

Reviewer Pack — Sturm Real-Root Count of GW Moment Matrix Bounds Max-CUT Sla...

Entry 6afd248521a0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 19:45:24 UTC

Sturm Real-Root Count of GW Moment Matrix Bounds Max-CUT Slack

Entry ID: 6afd248521a0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 19:45:24 UTC

1. Conjecture

Field A (mathematical branch): Real algebraic geometry — Hermite-Sturm real root counting on intervals via signed Hankel/subresultant signatures **Field B** (complexity object): Degree-2 Sum-of-Squares (Goemans-Williamson) pseudo-expectation moment matrix for max-CUT on 3-regular graphs

Statement:

Let G be a 3-regular graph on n vertices with Laplacian L_G ; obtain a degree-2 GW Gram matrix $M_G \in \mathbb{R}^{n \times n}$ by 80 steps of projected gradient ascent of $\langle L_G, V V^T \rangle / 4$ over rank- r unit-row matrices V with $r = \lceil \sqrt{2n} \rceil$ from a fixed deterministic init ($M_G = V V^T$ at convergence, all $\text{diag}(M_G) = 1$). Let $R(G) := (\# \text{ eigenvalues of } M_G \text{ lying in the open interval } (-1 + 1/\sqrt{n}, 1 - 1/\sqrt{n}))$, equivalently the positive index minus the negative index of the Hermite-Hankel form built from the first $2n$ traces $\text{tr}(M_G^k)$ restricted to that interval. Conjecture: for $n \in \{8, 10, 12, 14, 16, 18, 20, 25, 30, 35, 40\}$ and 30 i.i.d. uniform 3-regular graphs per n , $R(G)/n$ is concentrated within ± 0.07 of a universal constant $c \in [0.62, 0.80]$, and every instance with GW objective $\langle L_G, M_G \rangle / 4 \geq 0.878 \cdot \text{UB}(G)$ (where $\text{UB}(G) = |E|/2 + n \cdot \lambda_{\max}(L_G)/4$ is the spectral upper bound on MaxCut) satisfies $R(G)/n \geq 0.55$ — a single counterexample (high SDP ratio with $R/n < 0.55$, or low SDP ratio with $R/n \geq 0.80$) refutes the conjecture.

Rationale (proposer's reasoning):

The Hermite-Sturm count of moment-matrix eigenvalues in $(-1, 1)$ is a real-semialgebraic invariant of the SOS pseudo-distribution that is preserved under monomial relabelings and odd-degree real-affine reparametrizations, hence under simple polynomial reductions of CSPs. Tightness of the GW relaxation forces correlations to saturate to ± 1 along the cut direction, pushing M_G 's spectrum away from the open middle of $[-1, 1]$; conversely, slack leaves eigenvalues stranded in the interior, so the Hermite signature provides a structural witness rather than a truth-table single-number invariant. Because the invariant is built from real-spectral data of an SDP solution (not from polynomial-ring extensions of the Boolean cube), the construction sidesteps Razborov-Rudich, algebrization, and Karp-Lipton barriers while staying inside the Williams-style spectral lower-bound toolkit.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0806e6b00ff9e70d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 11 n -values $\times$ 30 random 3-regular graphs (330 instances), $R(G)/n$ is supported iff (i) a single constant $c \in [0.62, 0.80]$ exists with $\geq 85\%$ of instances within ± 0.07 of c , AND (ii) every instance with SDP ratio ≥ 0.878 has $R(G)/n \geq 0.55$. Any single violating instance falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Goemans-Williamson moment matrix eigenvalue distribution max-cut sum-of-squares - Hermite Sturm real root counting Hankel signature semidefinite relaxation - rank-constrained SDP max-cut 3-regular spectral bound pseudo-expectation

Top relevant hits considered: - [<http://arxiv.org/abs/2004.09833v2>] Expressing the largest eigenvalue of a singular beta F-matrix with heterogeneous hypergeometric functions - [<http://arxiv.org/abs/2604.0836>] Per-Shot Evaluation of QAOA on Max-Cut: A Black-Box Implementation Comparison with Goemans-Williamson - [<http://arxiv.org/abs/2112.11354v4>] Warm-Started QAOA with Custom Mixers Provably Converges and Computationally Beats Goemans-Williamson's Max-Cut at Low C_i - [<http://arxiv.org/abs/0812.4567v1>] On degenerate Hamburger moment problem and extensions of positive semidefinite Hankel block matrices - [<http://arxiv.org/abs/2108.02009v3>] Isolation Intervals of the Real Roots of the Parametric Cubic Equation and Improved Complete Root Classification - [<http://arxiv.org/abs/1104.1362v3>] Root Refinement for Real Polynomials - [<http://arxiv.org/abs/2109.02238v1>] Optimal solutions and ranks in the max-cut SDP - [<http://arxiv.org/abs/2604.13971v1>] Max Cut with Small-Dimensional SDP Solutions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=11.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_3_regular(n):
    if n % 2 != 0:
```

```

        return None # Cannot form a 3-regular graph with an odd number of vertices

G = set()
while len(G) < n * 3 // 2:
    u, v = random.sample(range(n), 2)
    if (u, v) not in G and (v, u) not in G:
        G.add((u, v))

return G

def is_3_regular(G):
    degree_count = [0] * n
    for u, v in G:
        degree_count[u] += 1
        degree_count[v] += 1

    return all(d == 3 for d in degree_count)

def project_to_unit_sphere(M):
    r = sum(sum(row) ** 2 for row in M) ** 0.5
    if r == 0:
        return M
    return [[M[i][j] / r for j in range(len(M[0]))] for i in range(len(M))]

def qr_with_shifts(A, shift):
    n = len(A)
    Q = [[Fraction(1) if i == j else Fraction(0) for j in range(n)] for i in range(n)]
    R = A
    for k in range(n - 1):
        v = [R[i][k] for i in range(k, n)]
        norm_v = sum(x ** 2 for x in v) ** 0.5
        e_k = [(Fraction(1) if i == k else Fraction(0)) for i in range(n)]
        u = [v[i] / norm_v - (sum(v[j] * Q[j][k] for j in range(k, n))) * e_k[k] for i in range(k, n)]
        Q_k = [[Fraction(1) if i == j else Fraction(0) for j in range(n)] for i in range(n)]
        Q_k[k][k] = u[0]
        Q_k[k + 1][k] = -u[1]
        Q_k[k][k + 1] = u[1]
        Q_k[k + 1][k + 1] = u[0]
        R = [[sum(Q[i][l] * R[l][j] for l in range(n)) for j in range(k, n)] for i in range(k, n)]
        A = [[sum(Q[i][l] * A[l][m] for l in range(n)) for m in range(n)] for i in range(n)]
    return Q, R

def compute_eigenvalues(M):
    n = len(M)
    Q, R = qr_with_shifts(M, 0)
    eigenvalues = [R[i][i] for i in range(n)]
    return eigenvalues

def compute_traces(M, k):
    n = len(M)
    trace = 0
    for i in range(n):
        for j in range(i + 1, n):
            trace += M[i][j] ** (2 * k)
    return trace

```

```

def generate_gw_gram_matrix(G, r):
    n = len(G)
    V = [[Fraction(1) / math.sqrt(n)] * n]
    for _ in range(r - 1):
        next_row = [sum(V[-1][i] * G[i][j] for i in range(n)) for j in range(n)]
        norm_next_row = sum(x ** 2 for x in next_row) ** 0.5
        V.append([x / norm_next_row for x in next_row])
    return [[sum(V[i][k] * V[j][l] for k in range(r)) for l in range(r)] for i in range(r)]

def compute_hankel_signature(M, n):
    w = [Fraction(1) if -1 + 1 / math.sqrt(n) < x < 1 - 1 / math.sqrt(n) else Fraction(0) for x in range(n)]
    H = [[sum(M[i][j] ** (k + l) * w[k] * w[l] for k in range(n) for l in range(n)) for j in range(n)]
    return H

def compute_maxcut(G):
    n = len(G)
    maxcut = 0
    for mask in range(1, 2 ** n):
        cut_size = sum(1 for i in range(n) if (mask >> i) & 1)
        if cut_size == 0 or cut_size == n:
            continue
        cut_value = sum(G[i][j] for i in range(n) for j in range(i + 1, n) if ((i >> j) & 1) ^ ((j >> i) & 1))
        maxcut = max(maxcut, cut_value)
    return maxcut

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [8, 10, 12, 14, 16, 18, 20, 25, 30, 35, 40]
    results = []
    for n in n_values:
        G = generate_random_3_regular(n)
        if G is None:
            continue

        r = math.ceil(math.sqrt(2 * n))
        M_G = generate_gw_gram_matrix(G, r)
        M_G = project_to_unit_sphere(M_G)

        eigenvalues = compute_eigenvalues(M_G)
        R_G = sum(1 for e in eigenvalues if -1 + 1 / math.sqrt(n) < e < 1 - 1 / math.sqrt(n))
        R_G_n = R_G / n

        H_G = compute_hankel_signature(eigenvalues, n)

        maxcut = compute_maxcut(G)
        UB_G = len(G) / 2 + n * max(eigenvalues) / 4
        SDP_ratio = <L_G, M_G> / (4 * UB_G)

        results.append({
            "metric_name": "R_G_n",
            "metric_value": R_G_n,
            "instances_tested": 1,
            "conjecture_holds": abs(R_G_n - c_star) <= 0.07 and SDP_ratio >= 0.878,
            "counterexample": ""
        })

```

```

mean_R_G_n = sum(result["metric_value"] for result in results) / len(results)
std_R_G_n = (sum((result["metric_value"] - mean_R_G_n) ** 2 for result in results) / len(results)) ** 0.5
support_fraction = sum(1 for result in results if abs(result["metric_value"] - c_star) <= 0.07)

return {
    "seed": seed,
    "mean_R_G_n": mean_R_G_n,
    "std_R_G_n": std_R_G_n,
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_R_G_n = sum(result["mean_R_G_n"] for result in results) / len(results)
    std_R_G_n = (sum((result["mean_R_G_n"] - mean_R_G_n) ** 2 for result in results) / len(results)) ** 0.5
    support_fraction = sum(1 for result in results if abs(result["mean_R_G_n"] - c_star) <= 0.07)

    if support_fraction >= 0.85:
        print(f"RESULT: SUPPORTED mean={mean_R_G_n} std={std_R_G_n} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture"])
        print(f"RESULT: FALSIFIED counterexample=\"SDP_ratio < 0.878\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a8a5c97.py", line 158, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a8a5c97.py", line 127, in run_trial
    G = generate_random_3_regular(n)
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a8a5c97.py", line 45, in generate_random_3_regular
    if is_3_regular(G):
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a8a5c97.py", line 23, in is_3_regular
    for u, v in G:
        ~~~~
ValueError: not enough values to unpack (expected 2, got 1)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed in `generate_random_3_regular` with a `ValueError` before producing any data, so no instances were evaluated against the pre-registered criterion. | next: Fix the graph generator's edge unpacking bug (iterate edges as tuples, e.g., `G.edges()`) and rerun the full 330-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	159586
2	preregistration	claude_max	opus	0	0	8100
3	novelty	claude_max	opus	0	0	3712
4	novelty	claude_max	opus	0	0	9829
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	171225
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	250417
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	254751
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	160563
9	judge	claude_max	opus	0	0	9310

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1027493 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/6afd248521a0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6afd248521a0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6afd248521a0.tar.gz` (if generated)